Dot Net MAUI:

Integrating Web Pages on Cross-Platform Applications

Jeremy Roalef

CITA 355: Advanced App Mobile Web Development

Professor Elfituri

Final Project

5/12/2026

Contents

Introduction

In a previous project, a web page was developed using local services to build a small student website. Students were able to create accounts, log in, take exams, and search the web page's database for results, all of which was made possible using PHP, HTML, and CSS. This project aims to expand the student website to the world of applications using Microsoft Visual Studio's .Net MAUI platform, ultimately allowing the student website to be used as an application on various platforms, including Windows, Android, and others.

Integrating .Net MAUI applications into the pre-existing web page system will require the development of a custom API system. The goal of this API is to allow direct communication from the MAUI application to the web page, and vise versa. In result, the API will allow our MAUI application to communicate with the web page's database, all the while the web page will remain unchanged and working as normal.

What is .Net MAUI

.Net MAUI is a cross-platform framework designed by Microsoft with the goal of allowing software developers to create applications across many platforms. It mimics the structure of websites, using XAML (Extensible Application Markup Language) in place of HTML, and C# in place of PHP. Additionally, .Net MAUI has various containers that mimic CSS elements, allowing developers to create an application that closely resembles web pages.

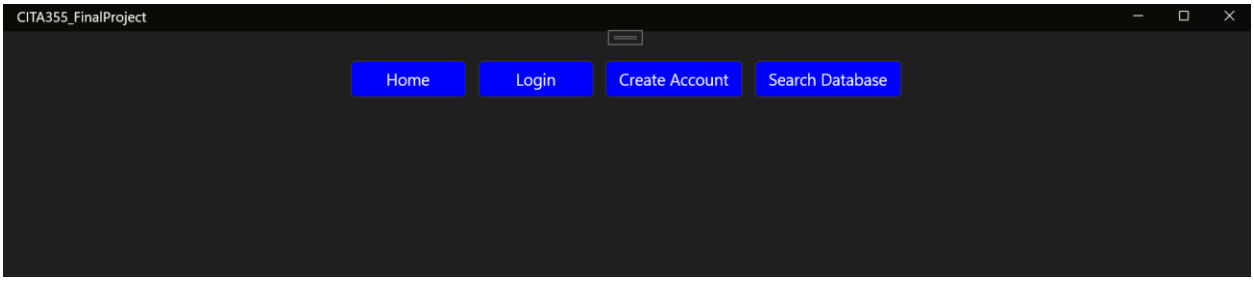
.Net MAUI uses C#, a highly supported object-oriented programming language, to handle the logistics of the application. C# acts as a developer-friendly, highly customizable programming language which helps break down the application's logic.

Application Overview

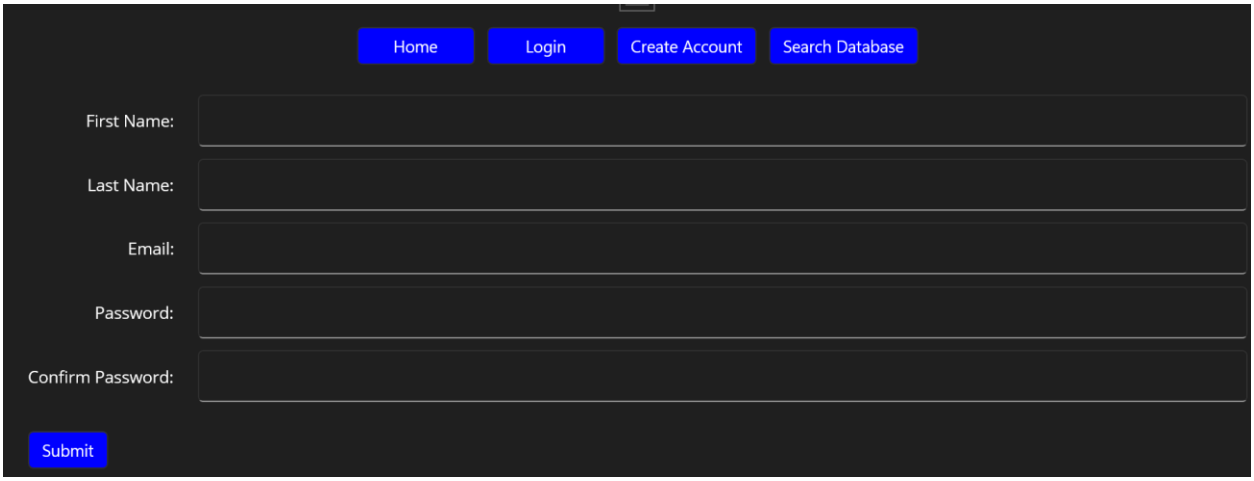
The application can be broken down into three systems: the content pages, the front-end API, and the back-end API

The Content Pages

The content pages make up the visuals and functionality of the application pages. The content pages, then, is anything that the user can see. For example, take the following landing page:



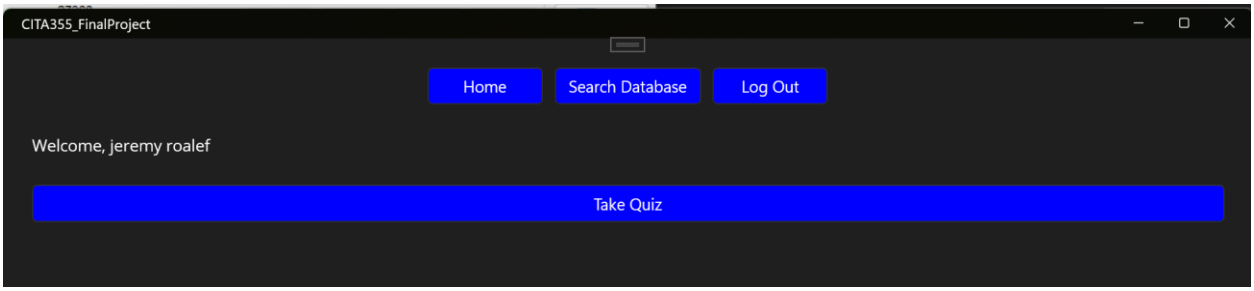
The user is prompted with four buttons, one to go home, to log in, to create an account, and to search the database. The user can click any of these four buttons, and they will be taken to a new content page. Say the user clicks the create account button. They will be prompted with the following content page:

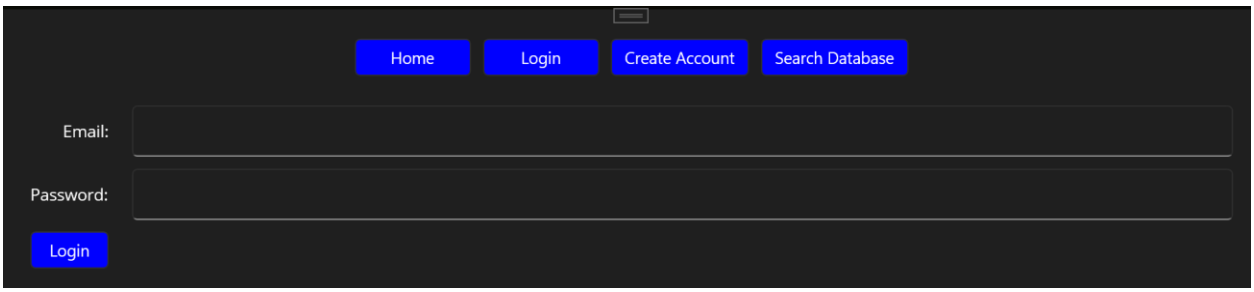


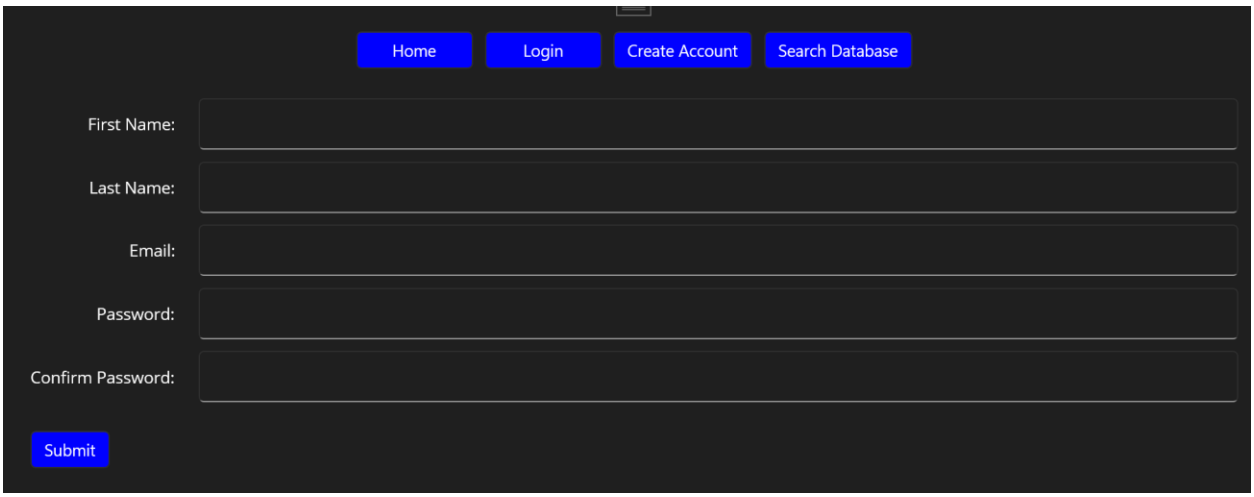
Now, the user can fill out their account information and submit it to the front-end API. Each content page makes up a subsection of the application. The list of all pages include:

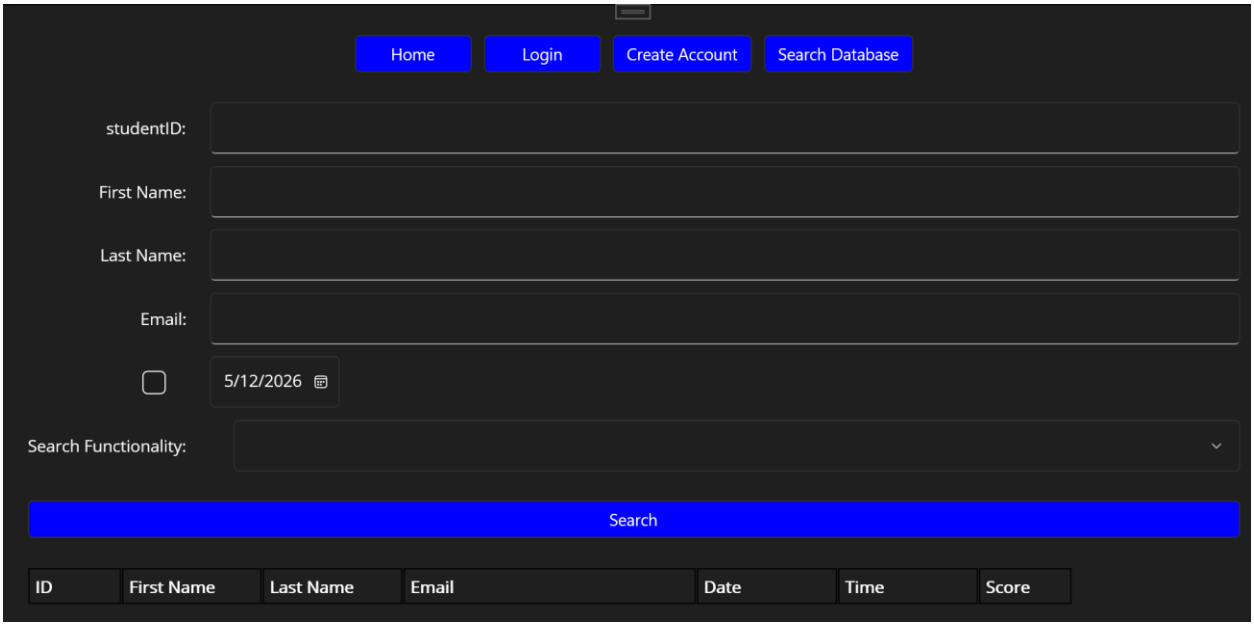
- The main page: Acts as the central hub for application navigation. It controls the logistics on whether the user can take an exam, log into or out of an account, or search the database.

- The login page: Acts as a form for the user to log in using their account credentials. Upon login, the user data is gathered from the database and stored locally in a session data class.
- The create account page: Acts as a form for the user to create an account to be stored in the database. Upon creation, the user is then logged in as that account and brought back to the home page, where they may now take the quiz.
- The search database page: Acts as both a form and a table view of database search queries. The user can query information from the database by many filters as prompted in the form. Upon submission, a table view will be created below the form for the user to see the results of their custom query.
- The exam page: Acts as a form for the exam . The user can take the exam and submit it to the database, receiving their final score after submission.



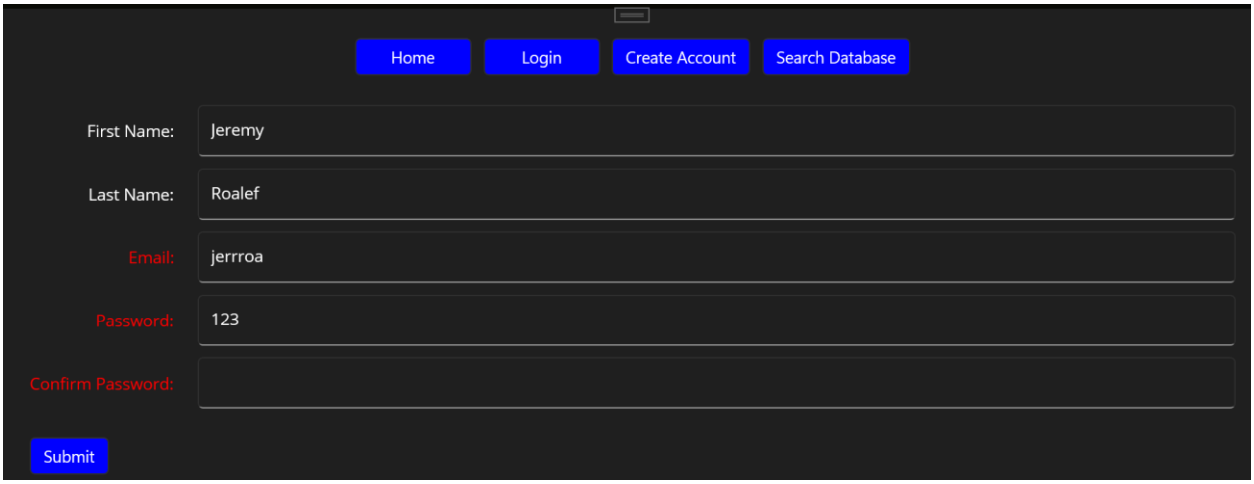






Content Page Input Validation

User input on the content pages must be validated before being submitted to the API system. Additionally, the content pages must validate the API calls for errors or API failures. Thus, every possible user input is validated before leaving the content page where the input was created. For example, the create account page validated every input field for a proper value before being submitted to the back-end API.



```
0 references
async void OnButtonSubmitClicked(object sender, EventArgs e)
{
    bool isValidFirstName = IsValidFirstName();
    bool isValidLastName = IsValidLastName();
    bool isValidEmail = IsValidEmail();
    bool isValidPassword = IsValidPassword();

    //Return if any input was invalid
    if (!isValidFirstName || !isValidLastName || !isValidEmail || !isValidPassword) return;

    //Craete the student object
    Student student = new Student(
        entryFirstName.Text,
        entryLastName.Text,
        entryPassword.Text,
        entryEmail.Text
        );

    try...
}
```

When invalid input is found, the content page prevents the user from submitting their account to the database until they meet the page's requirements.

The Front-End APIs

The front-end API refers to the API structure developed on the front-end application. The front-end API can be summarized by the following structure:

1. Get the back-end API URL
2. Create the HTTPClient object
3. Create a container of data to send to the back-end API
4. Create FormURLEncodedContent object and an HTTPResponseMessage object
5. Convert the HTTPResponseMessage to a string
6. Try to deserialize the response message to an APIResponse object using JSON deserialization
7. Return the APIResponse object to whoever called it.

```
1 reference
public static async Task<APIResponse> AddStudent(Student student)
{
    //URL to add new account
    string url = "http://localhost/CITA_355/Project2/processNewAccountAPI.php";

    //Create the HTTP Client
    HttpClient httpClient = new HttpClient();

    //Dictionary of data to pass to the web page
    Dictionary<string, string> accountData = new Dictionary<string, string>()
    {
        {"firstName", student.firstName },
        {"lastName", student.lastName },
        {"email" , student.email },
        {"password", student.password}
    };

    //Send the information to the web page
    FormUrlEncodedContent content = new FormUrlEncodedContent(accountData);
    HttpResponseMessage response = await httpClient.PostAsync(url, content);

    string responseAsString = await response.Content.ReadAsStringAsync();



    APIResponse? apiResponse = JsonSerializer.Deserialize<APIResponse>(responseAsString);

    //Return the response
    return apiResponse;
}
```

This seven-step process is used for every API call made from the front-end API to the back-end API. It's important that the front-end API tries to deserialize the returned to an API response. The API response class allows code execution to be executed based on the success status of the API response.

```
25 references
internal class APIResponse
{
    5 references
    public bool Success {  get; set; }
    4 references
    public Student Student { get; set; }
    9 references
    public string Message { get; set; }

    //Parameterless constructor needed for JSON deserialization
    //Just remember that...
    2 references
    public APIResponse()
    {

    }

    0 references
    public APIResponse(bool success, Student student, string message)
    {
        this.Success = success;
        this.Student = student;
        this.Message = message;
    }
}
```

Additionally, the API response system was engineered such that it could expand to add additional types of responses. This was achieved through an inheritance structure where the API response class is the base class. There are two subclasses that were used in this project API exam response and API search response.

```
5 references
internal class APIExamResponse : APIResponse
{
    1 reference
    public float Score {  get; set; }

    0 references
    public APIExamResponse() : base()
    {

    }
}
```

```
5 references
internal class APISearchResponse : APIResponse
{
    1 reference
    public List<SearchResult> Results { get; set; }

    0 references
    public APISearchResponse() : base()
    {

    }
}
```

These subclasses allowed me to personalize what I returned from the back-end API while keeping the structure clean and easy to read on the front-end API and application. It's also very scalable for future API response types and subtypes.

The Back-End API

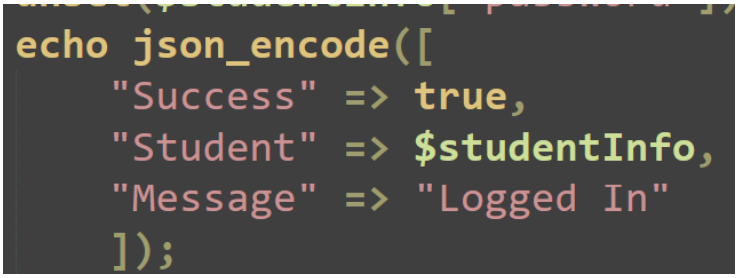
The back-end API is where the web page exists. The back-end API is the PHP-only pages that allows front-end applications to send API calls and receive JSON messages. Each back-end API can be summarized as the following:

1. Receive the input from the front-end API through the POST superglobal array
2. Validate all input from the POST array before performing the API request
3. Try to perform the API request, and handle exceptions and bad errors
4. Return an associative array back to the front-end API using JSON encoding

```php
if ($_SERVER['REQUEST_METHOD'] == 'POST'){
    $email = $_POST['email'] ?? '';
    $studentID = $_POST['studentID'] ?? '';
    $passwordString = $_POST['password'] ?? '';

    //Insert new student account into the database if the account values are not defaulted
    if ($studentID !== '' && !empty($passwordString)){
        LoginByStudentID($pdo, $studentID, $passwordString);
    }
    else if (!empty($email) && !empty($passwordString))
    {
        LoginByEmail($pdo, $email, $passwordString);
    }
    else{
        //One or more values were not properly set
        echo json_encode([
            "Success" => false,
            "Student" => null,
            "Message" => "data submitted was not properly submitted"
            ]);
    }
}
```

The back-end API ensures that every exit point of the API returns a JSON-encoded message. This ensures that the back-end API always communicates its status back to the front-end API, and then the front-end API can communicate its status back to the front-end application. Each json-encoded message will appear similar to the following:



The message above communicates directly with the API response class. However, variations of this exist in other back-end API calls, and those communicate directly to the subclasses of the API response base class. This system is how data can be send to the back-end API and how that back-end API can send its status back to the application.

Tools & Technologies Used

Below is a brief list of the tools and technologies used for this project:

- Microsoft Visual Studio 2022
- .Net MAUI framework
- Programming/Markup Languages
    - XAML
    - C#
    - PHP
- GitHub
- ChatGPT (for some parts of the project)

How Large Language Models (LLMs) Were Used in This Project

The use of LLMs, like ChatGPT, were used to expediate the development process for this project. LLMs were primarily used to quicken the code analysis and debugging process. When I write code, I will make many minor mistakes that add up over time. Due to time constraints, spending hours to days debugging my project system was not feasible. Thus, I relied on LLMs to analyze my code and find any issues present. For example, take the following prompt:

> "I found the error in my php. I am now getting responses again. However, I received an error thrown from my catch statement saying 'the JSON value could not be converted to System.Single'. Here's what I have on my PHP. What could cause this to happen? I'm storing the score as a float: [PHP file insertion]"

The issue I experienced was a data type mismatch. While I could've figured this out on my own without the use of LLMs, the error I received gave no direction towards the culprit of the thrown exception. Navigating the problem without LLMs could've taken many hours. However, ChatGPT was able to pinpoint the exact location in my PHP file where I was receiving the data type mismatch, allowing me to fix the thrown exception rapidly.

Where LLMs Failed

While LLMs are a great tool to expediate development processes, they frequently output misleading or misguided information that must be ignored. For example, many of my APIs rely on fetching results from my database. Whenever I feed a prompt to a LLM with my fetch code, it throws many unwanted paragraphs about the importance of using fetch associative over the base fetch call. Another example is when it tried to talk me out of developing an inherited API response system because the system would be "over-

engineered” and I should keep the system “simple”. I only prompted whether my idea would work, not whether or not I should use it:

> “The code worked wihtout problem. Now, I want to expand my APIResponse class to allow me to send specific exam responses. Right now, the APIResponse class stores the student object, the success state of the API, and the message. I was wondering if I could create a subclass, say APIExamResponse, that can also send back to my program the exam score. If I use an inheritance structures, and change JSON deserialization to APIExamResponse, would this work?”

While these are minor failures in a pool of successful responses from LLMs, these prompts reveal how LLMs are designed to create user dependencies on their prompts by giving you misguided information or alternatives that go against what you had originally envisioned. While LLMs are great for expediating processes, using them requires active rejection of poor advice or advice that goes against the developer’s system. To conclude, use LLMs with caution.

Lessons Learned

I took it upon myself to complete this project independently. I have four years of experience working in group projects in SUNY Morrisville, and I was hardly given the chance to complete projects independently. I've been a firm believer for a while now that I could develop my projects better if I did it independently. Completing this project was not only about whether I could finish the project requirements and get a passing grade. It was a test to see if my beliefs were valid.

After finishing this project, I believe my beliefs are more valid than before; I excel more when working in independent projects than group projects. I had full control over the development process and didn't have to rely on others to submit their deliverables on time. I could develop systems without the concern of handing it over to someone and them not knowing what they're looking at. I could debug my code faster because I was the sole author of it all. And I didn't have to pass my ideas through others, and vise versa, which would take time away from actually developing the project.

I know that I learned the wrong lessons for what SUNY Morrisville was preparing me for. However, I've been in so many group projects now where I knew I could do it better alone. At a certain point I think it's worth considering that maybe I just do better when working alone. Maybe I simply don't do well in group environments. This project was another point on that graph, trending towards what I've always felt to be true.

Reflection

Had I More Time, What Would I Do Differently?

I would not do anything differently. I developed my project such that it could scale easily as more requirements were added. For the purposes of this assignment, perhaps I could have sacrificed building scalable systems to meet all the project requirements. However, I am willing to lose points on my project if it means I get to explore the development of new systems.

What Aspects of The Application Would You Improve or Redesign?

I would primarily change how the project looks visually. I did not spend much time on the visual aesthetic of the page, partly because I didn't know how, but mainly because I chose to neglect it. I really like to build systems more than sit there trying to find the right color that fits the given element.

I would also redesign my search database table. The setup I used had one monolithic table of student and quiz information. For comparison, the web page was able to dynamically build the table using only the information selected from the given query. In other words, when the web page wanted to view quiz information only, the table only showed columns for quiz information.

Would I Choose .Net MAUI Again?

No, I would not choose .Net MAUI again. I do think .Net MAUI is well-developed. However, I know another platform that I am much more comfortable with, and it's much more integrated with the field of game programming; Unity. While Unity is primarily known for game development, it is also capable of developing 2D applications just like any other system and being cross-platform. You could even develop the web application in Unity and export your project as a WebGL build.

I am much more comfortable with Unity's entity-component system and parent-child object hierarchy than other systems. I am also well-versed in Unity's event execution order and the MonoBehaviour script runtime. Unity also provides many quality-of-life features that are grossly missing from other cross-platform frameworks. Most notably, Unity's UI and UX is extremely user-friendly, and Unity has many services that allow applications to go

from single-user applications to multi-user applications. There's also a lot more you can do to customize effects, logistics, and object lifetime that other frameworks hide or miss.

A quick Google search suggests that Unity can also handle web client connections via the built-in UnityWebRequest API. If I were to build a website/database system again, I would be very interested in using Unity to build applications and connect to web pages/databases. I also think this would be great if this course let students use Unity to develop the front-end application instead of .Net MAUI, especially since many students have expressed struggles using Unity's UI system in the past. This could be a great way to help game programming students learn more about Unity's UI system.

In conclusion, while I do see .Net MAUI as a valuable framework, it suffers from the same problems I've experience in many other development frameworks. I will keep .Net MAUI in the back of my mind, but I'm more interested in using Unity to do the same things I did in this project, just using Unity's user-friendly and customizable system over .Net MAUI's outdated system.

Future Work

Enhancements That Could be Made

While developing the application, I felt that I could have cached the user's session data to preserve their application's state while the application was closed. I already explored the logistics for this in a previous lab. Essentially, I just use JSON files to write object data to files and read it back to my objects when the application is opened. This would be a great quality of life feature, and I already have the proof of concept.

I feel that the exam could be expanded to store quizzes in databases. My idea is that you could create teacher accounts, and teachers can go in, create an exam, and that exam is stored on the database. Students can then query that exam from the database and take it. This would require an overhaul of the existing database structure, and I excluded this from my project since it would require solving too many logistical problems in such a short timespan. Given more time, I'd love to add this to the project.

How Could This Project Be Expanded/Improved in a Real-World Context?

In a real-world context, the UI would need to be completely overhauled. The current UI does not live up to modern application standards, even for back-end systems meant only for users that are part of the company. If I were to sell my application to someone, I'd completely rework the UI to something modern.